Documentation

Members:
Abila, Anajoy Niones
Buerano, Sean Kurvie Gan
Regulacion, Roxanne Nargatan
Ronquillo, Jenmarc Muhi
Rubia, Kyla Shane Tarroma

Bachelor of Science in Information Technology
COMP 018: Database Administration
Prof. John Kenneth L. Timblaco

PawPaw: Veterinary Appointment and Pet Management System

GROUP 3

Members

Rubia, Kyla Shane Tarroma	Project Manager
Abila, Anajoy Niones	Front-end Developer
Buerano, Sean Kurvie Gan	UI/UX Designer
Ronquillo, Jenmarc & Rubia, Kyla Shane	Back-end Developer
Regulacion, Roxanne Nargatan	Documentation

Repository
https://github.com/rubiakylashane/GRP03_PAWPAW.git

Tracker link
https://docs.google.com/spreadsheets/d/1BClCa9rEs9o-D9Rp3tUZVhECipJ234zX/edit?usp=sharing&ouid=106478744751340697951&rtpof=true&sd=true

System Overview

Veterinary clinics may experience long waiting lines, inconsistent appointment scheduling, difficulty managing pet records, and errors when recording information. These problems can make it difficult for clinic staff to properly organize appointments and keep track of each pet's medical history. They can also cause delays and inconvenience for pet owners, especially when appointments are not properly recorded or managed.

A Veterinary Appointment and Pet Management System, Pawpaw, is developed to provide a centralized and organized way to manage appointments, pet information, medical records, services, and other clinic records. It can help reduce scheduling conflicts, minimize errors, and make important information easier to access. For veterinary clinics, the system can make daily tasks more organized, while pet owners can conveniently book appointments, check their appointment history, and keep track of their pets' records.

General Objective

The general objective of Pawpaw is to develop an organized and easy-to-use system that helps veterinary clinics manage appointments, pet information, medical records, and clinic services. It aims to reduce scheduling problems and errors while making clinic services more convenient for both the staff and pet owners.

- Provide a registration and login feature for pet owners.
- Allow pet owners to add, update, and manage their pets' information.
- Manage veterinarian information, including their specialization and details.
- Allow pet owners to book, reschedule, and cancel appointments.
- Help clinic staff manage and update appointment schedules and statuses.
- Store and manage pet medical records, including diagnosis, treatment, and vaccination.
- Manage veterinary services and their corresponding prices.
- Record and track the appointment history of each pet.
- Record prescriptions given during veterinary consultations.
- Record vaccination given to registered pets.
- Provide search and filter functions for easier access to clinic records.
- Generate basic dashboards about appointments, services, and veterinarians.

Target Users

The proposed Veterinary Appointment and Pet Management System is intended for the following users:

- Pet Owners - Register their pets, book and manage appointments, view medical records, vaccination records, prescriptions, and appointment history.
- Veterinarians - View their appointments, access pet information, and record diagnoses, treatments, prescriptions, and other medical details.
- Veterinary Clinic Staff or Receptionists - Manage appointments, pet owner records, services, and other daily clinic information.
- Veterinary Clinic Administrators - Manage users, veterinarians, services, records, and dashboard information.

Scope and Limitations

Pawpaw will provide a centralized platform for managing common veterinary clinic activities and pet records. The system will cover owner registration and login, pet registration, veterinarian management, appointment booking and management, pet medical records, veterinary services and pricing, appointment history, vaccination records, prescription records, search and filtering, and basic dashboard. Pet owners can use the system to manage their pets and appointments, while veterinarians and clinic staff can manage appointments and update relevant pet and clinic records.

The system will be limited to appointment and record management and will not replace the professional judgment or physical services of veterinarians. It will not provide automated diagnosis, emergency veterinary care, online veterinary consultation, laboratory testing, or medicine delivery. The system will also not include online payment processing or integration with external veterinary clinics, pharmacies, or insurance providers. Medical information and treatment records will be entered and managed by authorized clinic staff or veterinarians rather than being automatically generated by the system.

User Roles

This section defines the responsibilities, permitted functions, and access restrictions of each application user role.

Application Role	Responsibilities in the Application	Functions and Data Access	Intended Database Role
Administrator	Manages user accounts, system settings, and clinic operations.	Create, view, update, and delete user accounts as authorized; manage veterinary services; oversee appointments and records; view reports and audit logs.	app_admin_grp
Staff	Assists with appointment scheduling and daily clinic operations.	Create and view appointments; update appointment statuses; manage pet and owner information, and clinic services according to assigned permissions. Access to protected medical records is restricted.	app_runtime_grp

Veterinarian	Handles veterinary consultations and maintains pet medical information.	View assigned appointments and pet information; create and update medical records; record diagnoses, treatments, prescriptions, and vaccinations.	app_runtime_grp
Pet Owner	Manages personal information, registered pets, and veterinary appointments.	Register and manage personal pet records; book, reschedule, and cancel appointments; view appointment history and relevant medical records, prescriptions, and vaccination records for their own pets.	app_runtime_grp

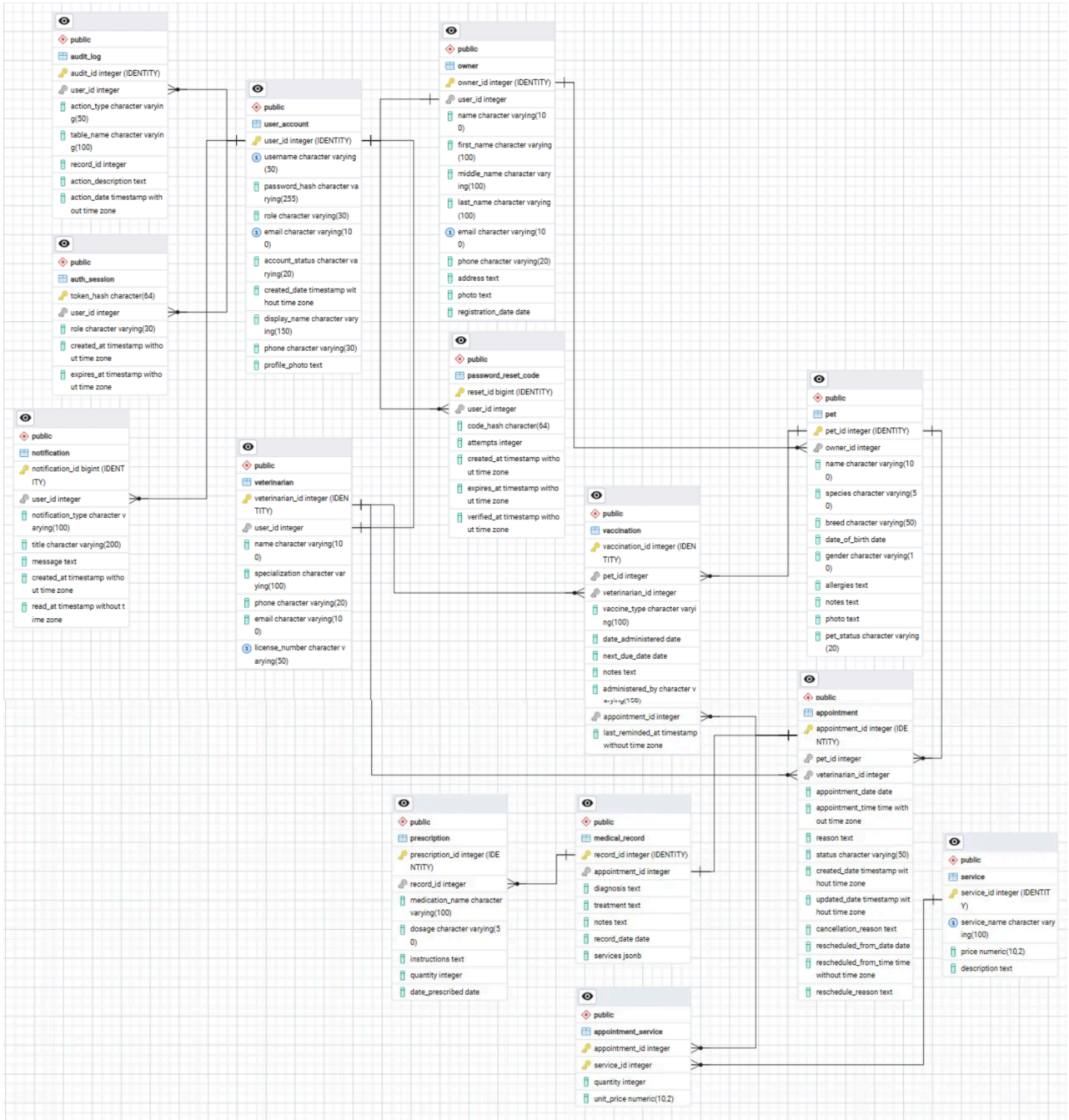
Development Progress Log

This log documents the project's development progress, including completed tasks, assigned team members, major milestones, and related Git commits.

Date	Milestone	Team Member(s) Responsible	Status / Remarks
Oct 18	Developed frontend and backend components for pet owner registration, pet profile management, appointment booking, appointment history, and medical record viewing. Includes initial API development and database integration.	Anajoy, Sean, Jenmarc	Completed
Oct 20	Developed frontend interfaces and backend functionality for appointment management, scheduling, medical records, prescriptions, and vaccination records. Includes API integration and database operations.	Anajoy, Sean, Jenmarc	Completed
Oct 22	Administrator interface and backend functionality for managing user accounts, veterinarians, services, and clinic information. Integrated frontend, backend APIs, and PostgreSQL database.	Anajoy, Sean, Jenmarc, Shane	Completed
Oct 24	System testing results, bug fixes, verification of database operations and user permissions, and updated technical documentation.	Roxanne, Shane	Completed
Oct 26	Finalized source code, database schema and SQL scripts, README and setup instructions, system documentation, development progress log, testing evidence, screenshots, and Activity 3 report.	Roxanne, Shane	Completed

Database Implementation

This section presents the Entity-Relationship Diagram (ERD), schema summary, migrations or SQL scripts used, seed data, and any changes from the approved design with reasons.



SQL Script

```
CREATE TABLE IF NOT EXISTS USER_ACCOUNT (
   user_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   username VARCHAR(50) NOT NULL UNIQUE,
   password_hash VARCHAR(255) NOT NULL,
   role VARCHAR(30) NOT NULL,
   email VARCHAR(100) UNIQUE,
   account_status VARCHAR(20) NOT NULL DEFAULT 'Active',
   created_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   display_name VARCHAR(150),
   phone VARCHAR(30),
   profile_photo TEXT,

   CONSTRAINT chk_user_role
      CHECK (role IN ('Pet Owner', 'Veterinarian', 'Staff', 'Admin')),

   CONSTRAINT chk_account_status
      CHECK (account_status IN ('Active', 'Inactive'))
);


CREATE TABLE IF NOT EXISTS OWNER (
   owner_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   user_id INT UNIQUE,
   name VARCHAR(100) NOT NULL,
   first_name VARCHAR(100),
   middle_name VARCHAR(100),
   last_name VARCHAR(100),
   email VARCHAR(100) UNIQUE,
   phone VARCHAR(20),
   address TEXT,
   photo TEXT,
   registration_date DATE NOT NULL DEFAULT CURRENT_DATE,

   FOREIGN KEY (user_id)
      REFERENCES USER_ACCOUNT(user_id)
      ON DELETE SET NULL
);


CREATE TABLE IF NOT EXISTS VETERINARIAN (
   veterinarian_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   user_id INT UNIQUE,
   name VARCHAR(100) NOT NULL,
   specialization VARCHAR(100),
   phone VARCHAR(20),
   email VARCHAR(100),
   license_number VARCHAR(50) UNIQUE,

   FOREIGN KEY (user_id)
      REFERENCES USER_ACCOUNT(user_id)
      ON DELETE SET NULL
);
```

```
CREATE TABLE IF NOT EXISTS SERVICE (
   service_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   service_name VARCHAR(100) NOT NULL UNIQUE,
   price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
   description TEXT
);

INSERT INTO SERVICE (service_name, price, description)
VALUES
   ('General Consultation', 500, 'A full check of your pet''s health with a veterinarian, including advice on diet and care'),
   ('Vaccine: Anti Rabies', 350, 'Core and booster vaccines, recorded in your pet''s history with the next due date.'),
   ('Vaccine: 5-in-1 DHPPL', 350, 'Core and booster vaccines, recorded in your pet''s history with the next due date.'),
   ('Vaccine: Feline 4-in-1', 350, 'Core and booster vaccines, recorded in your pet''s history with the next due date.'),
   ('Vaccine: Kennel Cough', 350, 'Core and booster vaccines, recorded in your pet''s history with the next due date.'),
   ('Deworming', 250, 'Treatment for common intestinal parasites, with a follow-up schedule when needed'),
   ('Grooming', 600, 'Bath, brush, nail trim, and ear cleaning for a clean and comfortable pet.'),
   ('Dental Cleaning', 1500, 'Scaling and polishing to keep teeth and gums healthy.'),
   ('Laboratory Test', 900, 'Blood and other lab work ordered by your veterinarian to support a diagnosis.')
ON CONFLICT (service_name) DO NOTHING;


CREATE TABLE IF NOT EXISTS PET (
   pet_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   owner_id INT NOT NULL,
   name VARCHAR(100) NOT NULL,
   species VARCHAR(50) NOT NULL,
   breed VARCHAR(50),
   date_of_birth DATE,
   gender VARCHAR(10),
   allergies TEXT,
   notes TEXT,
   photo TEXT,
   pet_status VARCHAR(20) NOT NULL DEFAULT 'Active',

   FOREIGN KEY (owner_id)
      REFERENCES OWNER(owner_id)
      ON DELETE RESTRICT,

   CONSTRAINT chk_pet_gender
      CHECK (gender IS NULL OR gender IN ('Male', 'Female', 'Unknown')),

   CONSTRAINT chk_pet_status
      CHECK (pet_status IN ('Active', 'Inactive')),

   CONSTRAINT chk_pet_birth_date
      CHECK (date_of_birth IS NULL OR date_of_birth <= CURRENT_DATE)
);

ALTER TABLE PET
   ADD COLUMN IF NOT EXISTS pet_status VARCHAR(20) NOT NULL DEFAULT 'Active';
```

```
ALTER TABLE USER_ACCOUNT
   ADD COLUMN IF NOT EXISTS profile_photo TEXT;

DO $$
BEGIN
   IF NOT EXISTS (
      SELECT 1 FROM pg_constraint
       WHERE conname = 'chk_pet_status'
         AND conrelid = 'pet'::regclass
   ) THEN
      ALTER TABLE PET
         ADD CONSTRAINT chk_pet_status CHECK (pet_status IN ('Active', 'Inactive'));
   END IF;
END $$;


CREATE TABLE IF NOT EXISTS APPOINTMENT (
   appointment_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   pet_id INT NOT NULL,
   veterinarian_id INT,
   appointment_date DATE NOT NULL,
   appointment_time TIME NOT NULL,
   reason TEXT,
   status VARCHAR(50) NOT NULL DEFAULT 'Scheduled',
   created_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   updated_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
   cancellation_reason TEXT,
   rescheduled_from_date DATE,
   rescheduled_from_time TIME,
   reschedule_reason TEXT,

   FOREIGN KEY (pet_id)
      REFERENCES PET(pet_id)
      ON DELETE RESTRICT,

   FOREIGN KEY (veterinarian_id)
      REFERENCES VETERINARIAN(veterinarian_id)
      ON DELETE RESTRICT,

   CONSTRAINT chk_appointment_status
      CHECK (status IN (
         'Pending',
         'Scheduled',
         'Approved',
         'Checked-in',
         'In Progress',
         'Completed',
         'Cancelled',
         'No-show'
      ))
);
```

```
CREATE TABLE IF NOT EXISTS APPOINTMENT_SERVICE (
   appointment_id INT NOT NULL,
   service_id INT NOT NULL,
   quantity INT NOT NULL DEFAULT 1 CHECK (quantity > 0),

   PRIMARY KEY (appointment_id, service_id),

   FOREIGN KEY (appointment_id)
      REFERENCES APPOINTMENT(appointment_id)
      ON DELETE CASCADE,

   FOREIGN KEY (service_id)
      REFERENCES SERVICE(service_id)
      ON DELETE RESTRICT
);


CREATE TABLE IF NOT EXISTS MEDICAL_RECORD (
   record_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   appointment_id INT NOT NULL UNIQUE,
   diagnosis TEXT NOT NULL,
   treatment TEXT,
   notes TEXT,
   record_date DATE NOT NULL DEFAULT CURRENT_DATE,
   services JSONB NOT NULL DEFAULT '[]'::JSONB,

   FOREIGN KEY (appointment_id)
      REFERENCES APPOINTMENT(appointment_id)
      ON DELETE RESTRICT
);


CREATE TABLE IF NOT EXISTS PRESCRIPTION (
   prescription_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   record_id INT NOT NULL,
   medication_name VARCHAR(100) NOT NULL,
   dosage VARCHAR(50) NOT NULL,
   instructions TEXT NOT NULL,
   quantity INT NOT NULL CHECK (quantity > 0),
   date_prescribed DATE NOT NULL DEFAULT CURRENT_DATE,

   FOREIGN KEY (record_id)
      REFERENCES MEDICAL_RECORD(record_id)
      ON DELETE CASCADE
);


CREATE TABLE IF NOT EXISTS VACCINATION (
   vaccination_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   pet_id INT NOT NULL,
   veterinarian_id INT,
   vaccine_type VARCHAR(100) NOT NULL,
   date_administered DATE NOT NULL,
```

```
    next_due_date DATE,
    notes TEXT,
    administered_by VARCHAR(150),
    appointment_id INT,
    last_reminded_at TIMESTAMP,

    FOREIGN KEY (pet_id)
        REFERENCES PET(pet_id)
        ON DELETE RESTRICT,

    FOREIGN KEY (veterinarian_id)
        REFERENCES VETERINARIAN(veterinarian_id)
        ON DELETE RESTRICT,

    FOREIGN KEY (appointment_id)
        REFERENCES APPOINTMENT(appointment_id)
        ON DELETE SET NULL,

    CONSTRAINT chk_vaccination_dates
        CHECK (
            next_due_date IS NULL
            OR next_due_date >= date_administered
        )
);


CREATE TABLE IF NOT EXISTS AUDIT_LOG (
    audit_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id INT,
    action_type VARCHAR(50) NOT NULL,
    table_name VARCHAR(100) NOT NULL,
    record_id INT,
    action_description TEXT,
    action_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id)
        REFERENCES USER_ACCOUNT(user_id)
        ON DELETE SET NULL,

    CONSTRAINT chk_audit_action_type
        CHECK (action_type IN (
            'INSERT', 'UPDATE', 'DELETE', 'LOGIN', 'LOGOUT'
        ))
);


CREATE TABLE IF NOT EXISTS NOTIFICATION (
    notification_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id INT NOT NULL,
    notification_type VARCHAR(100) NOT NULL,
    title VARCHAR(200) NOT NULL,
    message TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    read_at TIMESTAMP,

    FOREIGN KEY (user_id)
        REFERENCES USER_ACCOUNT(user_id)
        ON DELETE CASCADE
);


CREATE TABLE IF NOT EXISTS AUTH_SESSION (
    token_hash CHAR(64) PRIMARY KEY,
    user_id INT NOT NULL,
    role VARCHAR(30) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NOT NULL,

    FOREIGN KEY (user_id)
        REFERENCES USER_ACCOUNT(user_id)
        ON DELETE CASCADE,

    CONSTRAINT chk_auth_session_expiry
        CHECK (expires_at > created_at)
);

CREATE TABLE IF NOT EXISTS PASSWORD_RESET_CODE (
    reset_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id INT NOT NULL,
    code_hash CHAR(64) NOT NULL,
    attempts INT NOT NULL DEFAULT 0 CHECK (attempts >= 0),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NOT NULL,
    verified_at TIMESTAMP,

    FOREIGN KEY (user_id)
        REFERENCES USER_ACCOUNT(user_id)
        ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_password_reset_code_user_created
    ON PASSWORD_RESET_CODE (user_id, created_at DESC);
```

Seed Data

```
const ACCOUNTS = [
  { username: 'seed.owner', role: 'Pet Owner', email: 'seed.owner@pawpaw.test', name: 'Seed Owner' },
  { username: 'seed.staff', role: 'Staff', email: 'seed.staff@pawpaw.test', name: 'Seed Staff' },
  { username: 'seed.vet', role: 'Veterinarian', email: 'seed.vet@pawpaw.test', name: 'Seed Veterinarian' },
  { username: 'seed.admin', role: 'Admin', email: 'seed.admin@pawpaw.test', name: 'Seed Administrator'
}
];
```

Changes from Approved Design

The main changes involve additional profile information, expanded appointment workflows, vaccination tracking, authentication features, and database constraints.

1. Changes to existing tables

Table	Changes in Final schema.sql	Reason for Change
USER_ACCOUNT	Added display_name, phone, and profile_photo.	To support more complete user profiles and profile customization.
OWNER	Added first_name, middle_name, last_name, and photo.	To store more detailed owner information and profile photos.
VETERINARIAN	No changes to columns or their data types.	The original structure was retained because it supports the required veterinarian information.
SERVICE	No changes to table structure; added nine predefined veterinary services with prices and descriptions.	To make clinic services available immediately after database initialization.
PET	Added allergies, notes, photo, and pet_status. Added checks for gender, status, and birth date.	To support more detailed pet profiles, health information, and validation of pet data.
APPOINTMENT	Made veterinarian_id nullable. Added cancellation_reason, rescheduled_from_date, rescheduled_from_time, and reschedule_reason. Expanded appointment statuses.	To support appointments without an initially assigned veterinarian and provide more detailed cancellation, rescheduling, and appointment tracking.
APPOINTMENT_SERVICE	No structural changes.	The original many-to-many relationship between appointments and services was retained.
MEDICAL_RECORD	Added services with a JSONB data type and an empty-array default.	To store service-related information directly within a medical record in structured JSON format.

PRESCRIPTION	No structural changes.	The original prescription fields and relationship to medical records were retained.
VACCINATION	Made veterinarian_id nullable. Added administered_by, appointment_id, and last_reminded_at.	To support flexible vaccination documentation, appointment linking, and reminder tracking.
AUDIT_LOG	No structural changes.	The original audit log structure and action-type constraints were retained.

2. Newly added tables

New Table	Purpose	Reason for Addition
NOTIFICATION	Stores user notifications, including their type, title, message, creation time, and read time.	To support in-system notifications and track whether users have read them.
AUTH_SESSION	Stores authentication session information, including token hashes, associated users, roles, and expiration times.	To support session management and authentication security.
PASSWORD_RESET_CODE	Stores hashed password reset codes, attempt counts, verification times, and expiration times.	To support password recovery and verification.

All three tables reference USER_ACCOUNT using ON DELETE CASCADE, meaning their associated records are automatically removed when the referenced user account is permanently deleted.

3. Changes to constraints and database operations

Component	Initial SQL	Final schema.sql	Reason for Change
Table creation	CREATE TABLE	CREATE TABLE IF NOT EXISTS	To avoid errors when a table already exists.

Appointment status	Scheduled, Completed, Cancelled, No Show	Pending, Scheduled, Approved, Checked-in, In Progress, Completed, Cancelled, No-show	To support additional stages of the appointment workflow.
Pet gender	No specific validation	Allows Male, Female, Unknown, or NULL	To standardize gender values.
Pet status	Not included	Active or Inactive, defaulting to Active	To support pet status management.
Pet birth date	No date validation	Birth date must not be later than the current date	To prevent future birth dates from being entered.
Vaccination appointment	No appointment reference	Added nullable appointment_id with ON DELETE SET NULL	To link vaccinations to appointments while preserving vaccination records if an associated appointment is deleted.
Authentication session expiration	Not included	expires_at must be later than created_at	To ensure valid session expiration periods.
Password reset attempts	Not included	attempts must be zero or greater	To prevent negative attempt counts.
Password reset index	Not included	Added index on user_id and created_at DESC	To improve queries retrieving a user's password reset records by creation date.
Existing-table adjustments	Not included	Added ALTER TABLE and conditional constraint creation statements	To accommodate selected structural updates when working with existing tables.
Service initialization	Not included	Added service insertion with ON CONFLICT (service_name) DO NOTHING	To populate default services without duplicating existing service names.

4. Overall comparison

Category	Initial SQL	Final schema.sql
Total tables	11	14

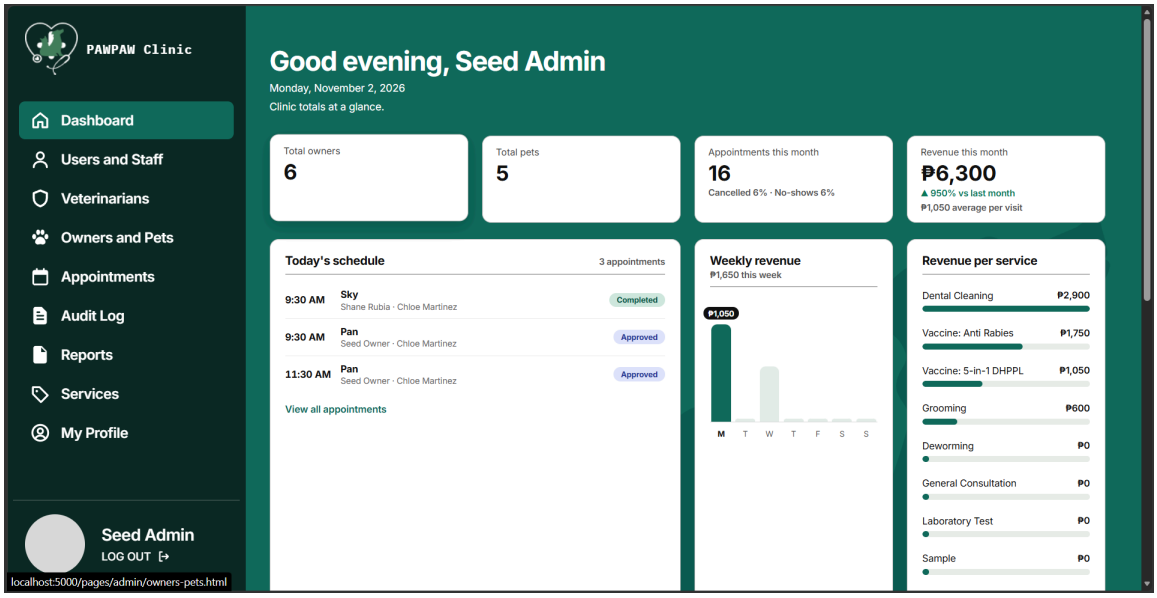
User profiles	Basic account and owner information	Additional profile fields and photos
Pet management	Basic pet details	Health notes, allergies, photos, and status
Appointment management	Four statuses	Eight statuses and rescheduling/cancellation details
Medical records	Diagnosis, treatment, and notes	Added structured service information
Vaccination management	Basic vaccination records	Appointment linking, administrator identification, and reminder tracking
Notifications	Not included	Included
Authentication sessions	Not included	Included
Password recovery	Not included	Included
Database initialization	Table creation only	Conditional table creation, selected alterations, initial service data, and an index

Application Evidence

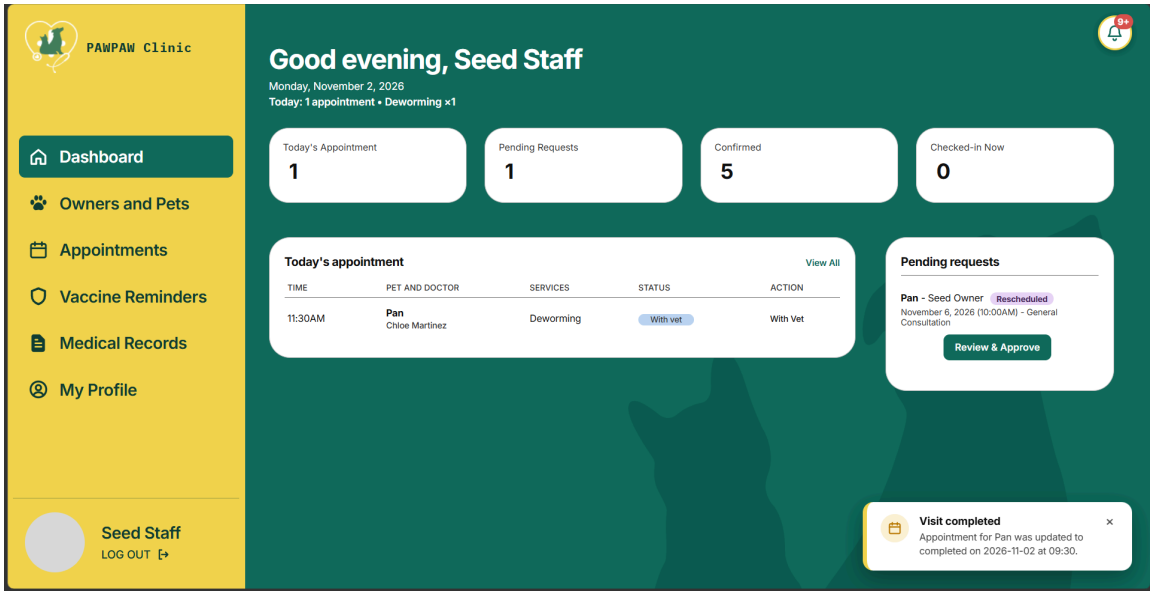
This section includes screenshots of the working system and results of the create, retrieve, update, and delete (CRUD) tests.

1. System overview and working application

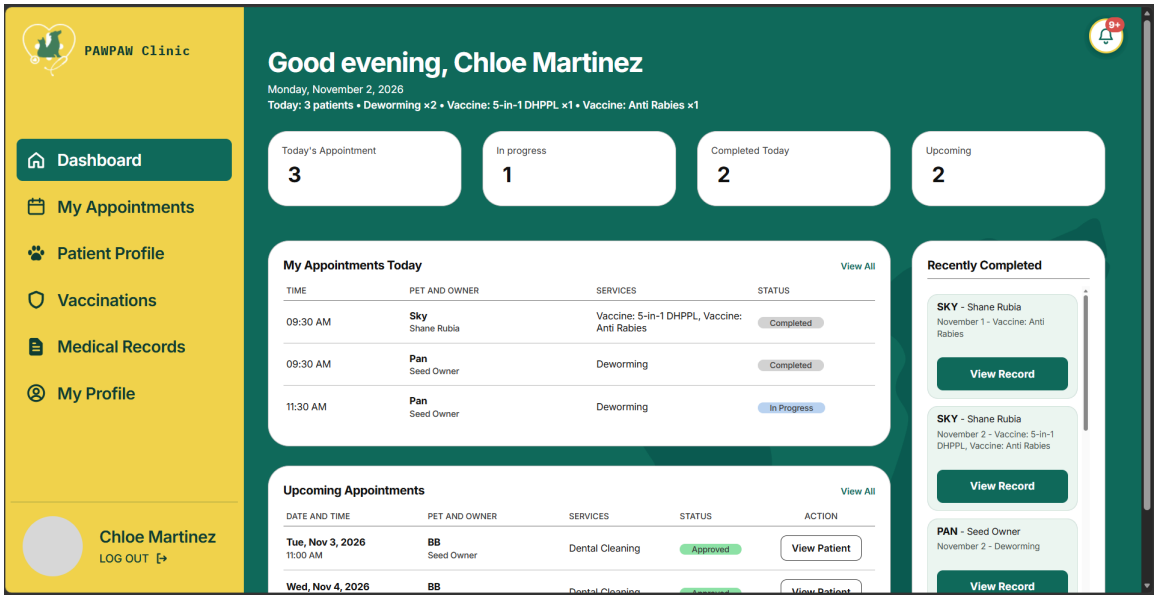
A. Admin Dashboard



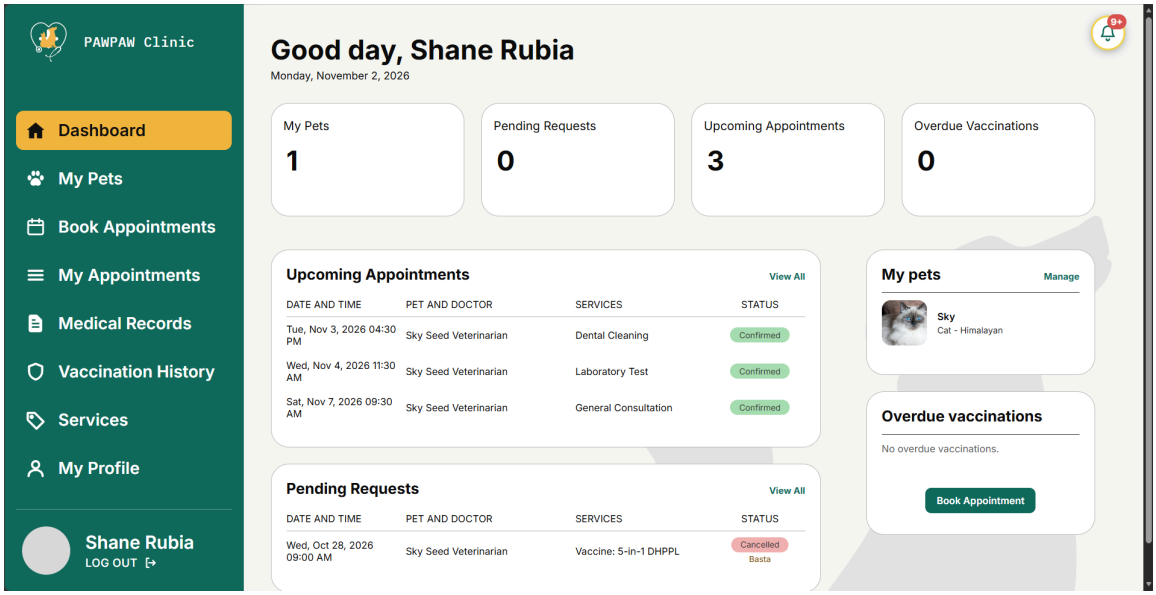
B. Staff Dashboard



C. Veterinarian Dashboard



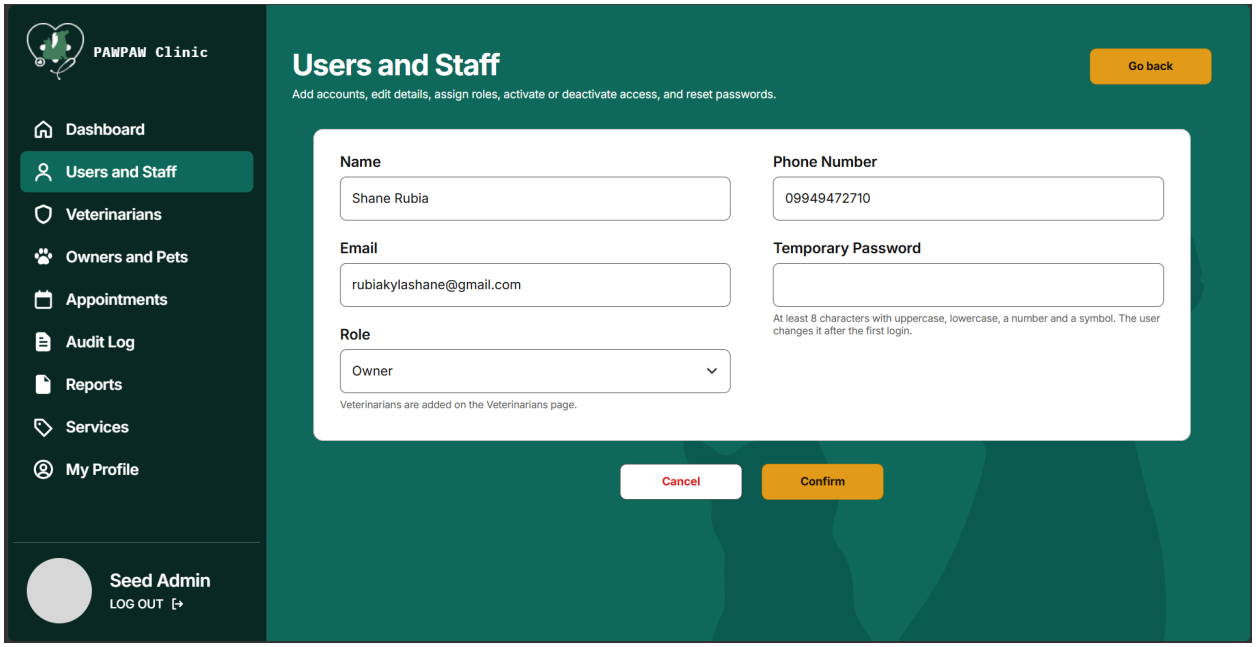
D. Owner Dashboard



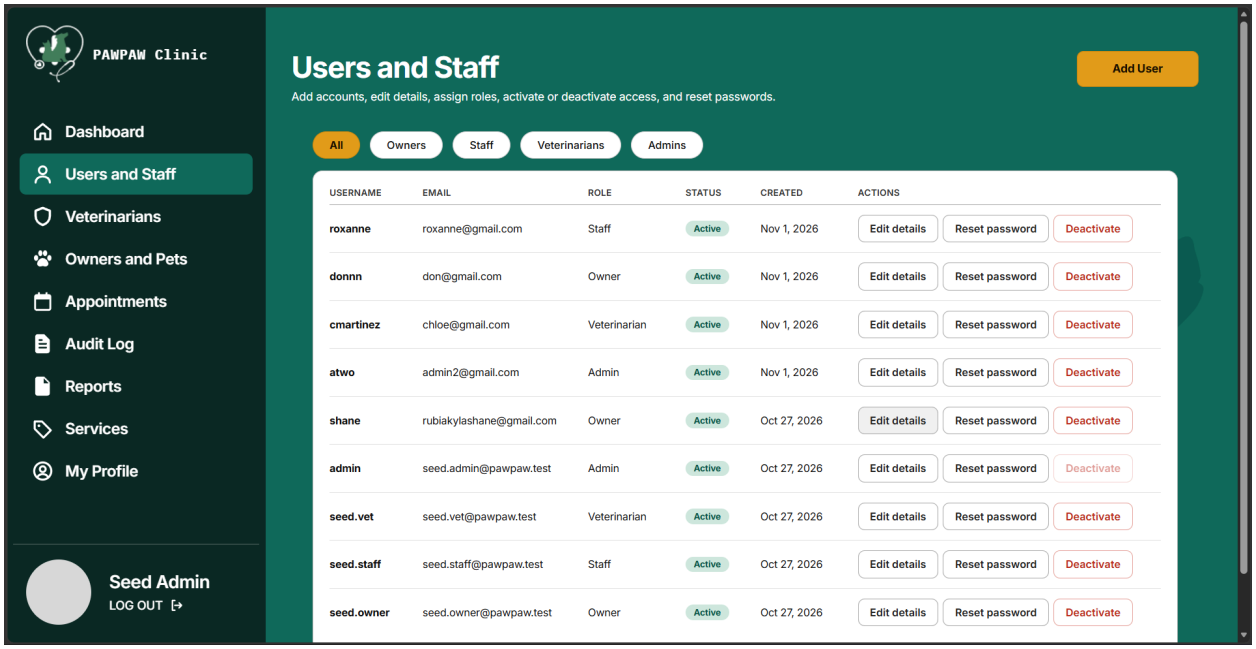
2. CRUD operation

A. Admin

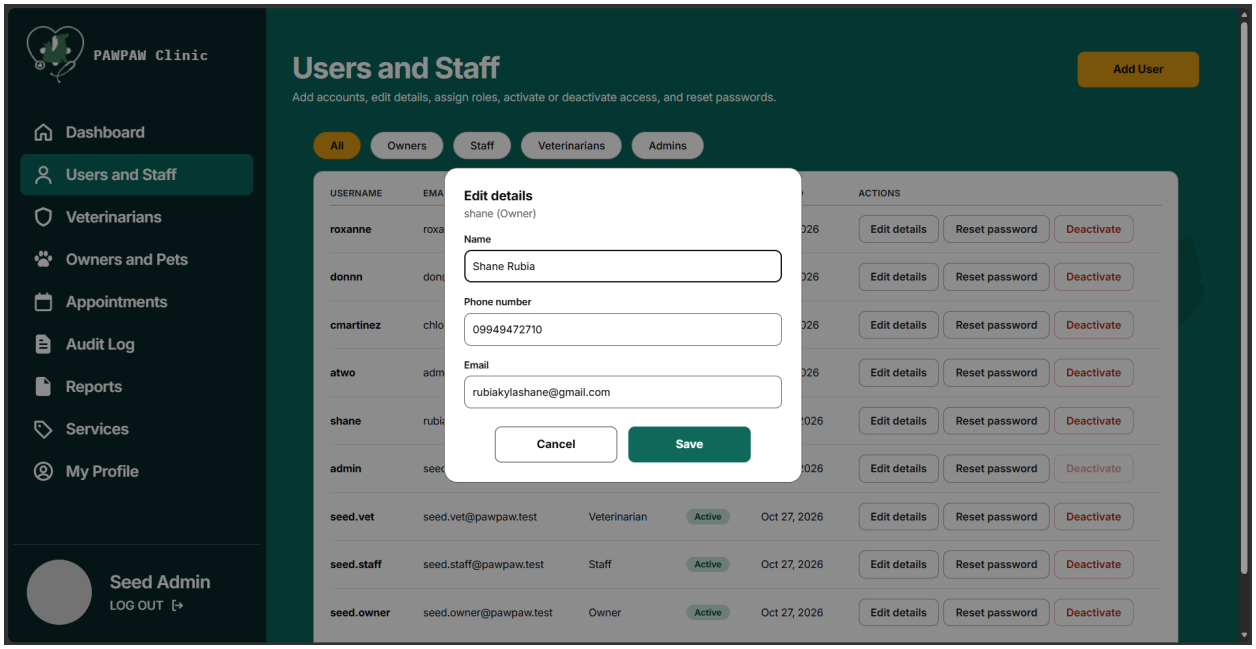
Create: Add a new user account.



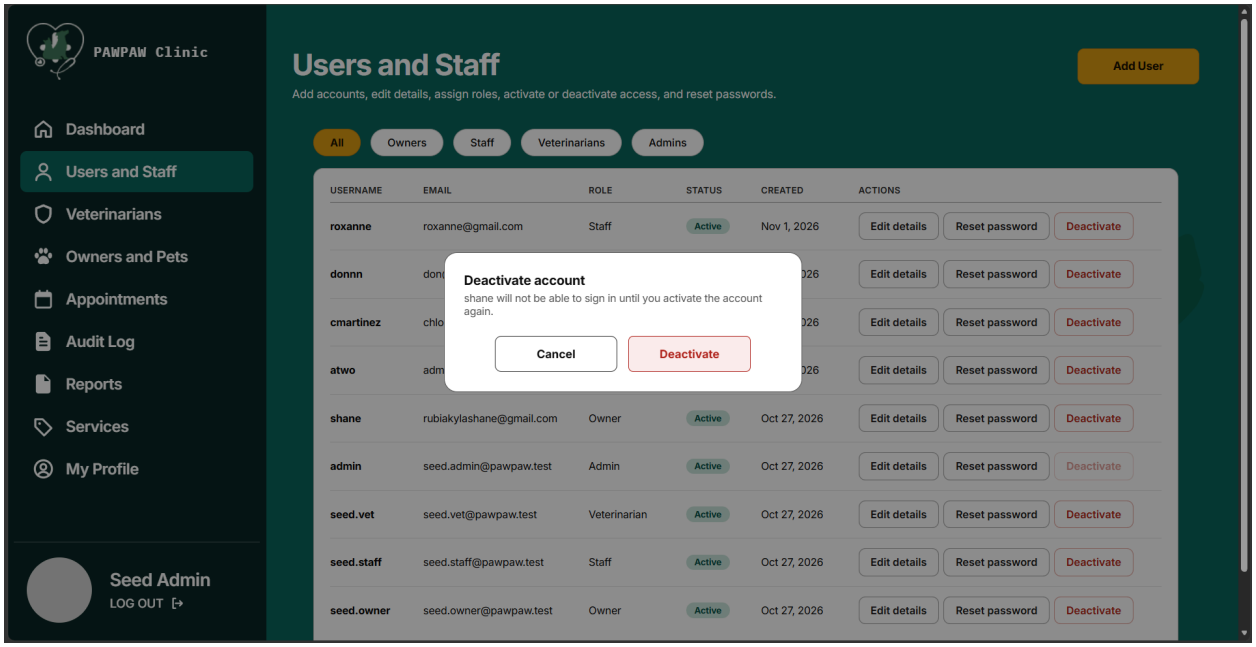
Read: View the list of registered users.



Update: Edit user information or role.

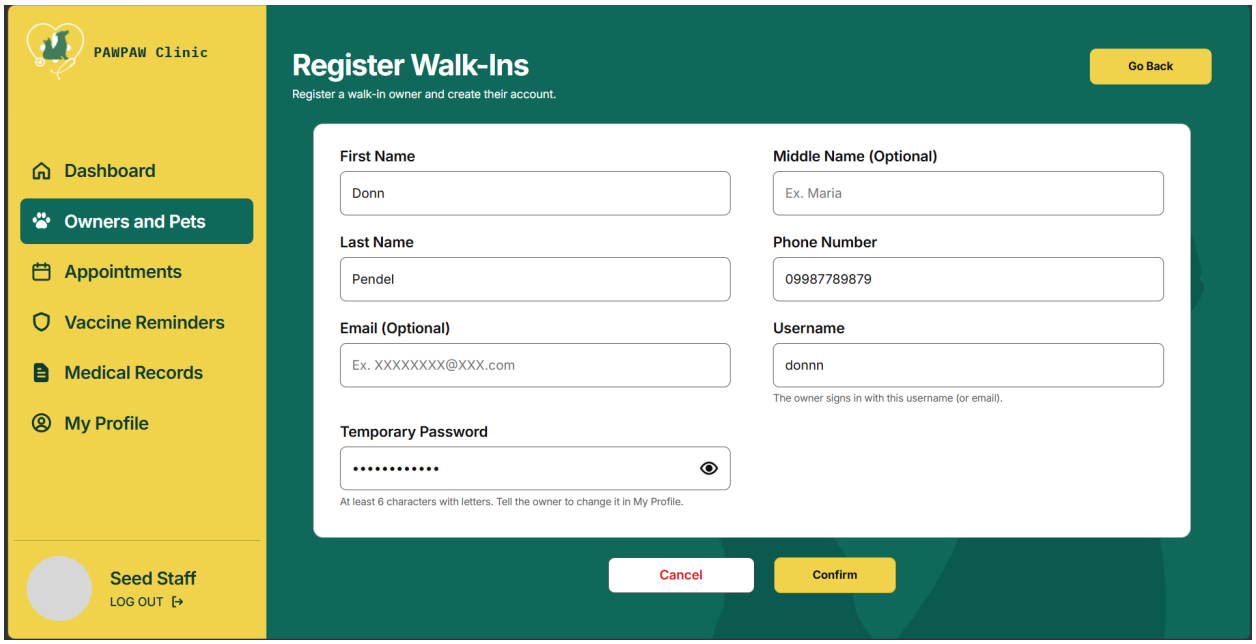


Delete: Deactivate a user account.

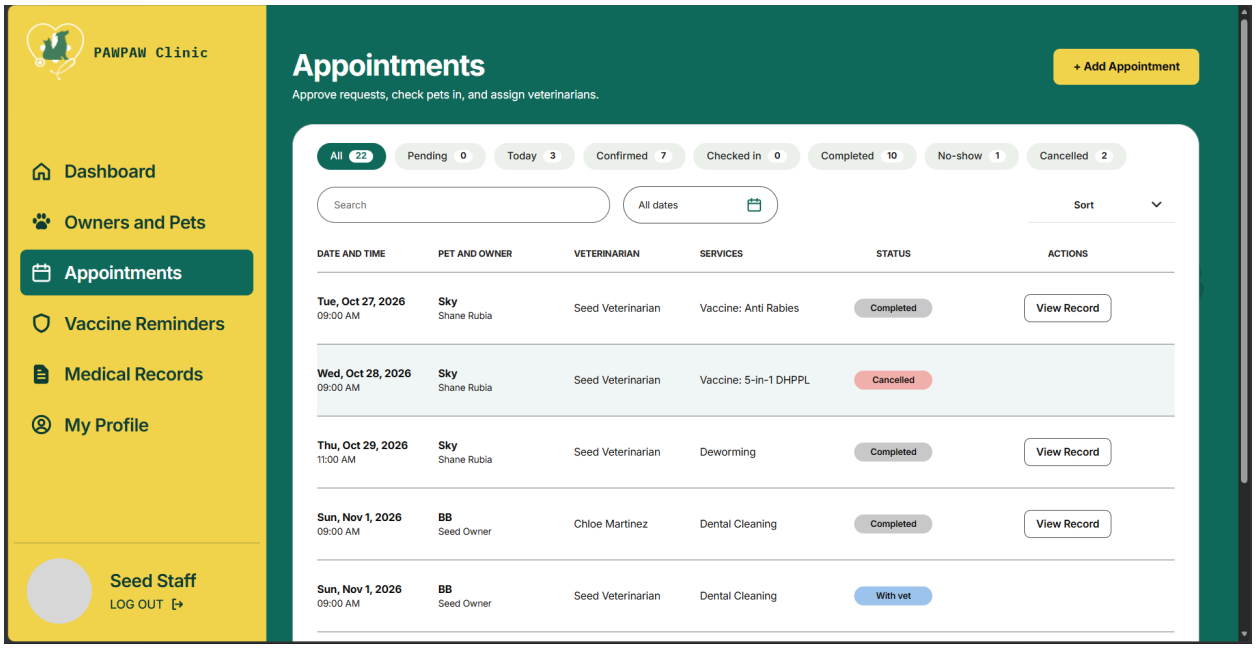


B. Staff

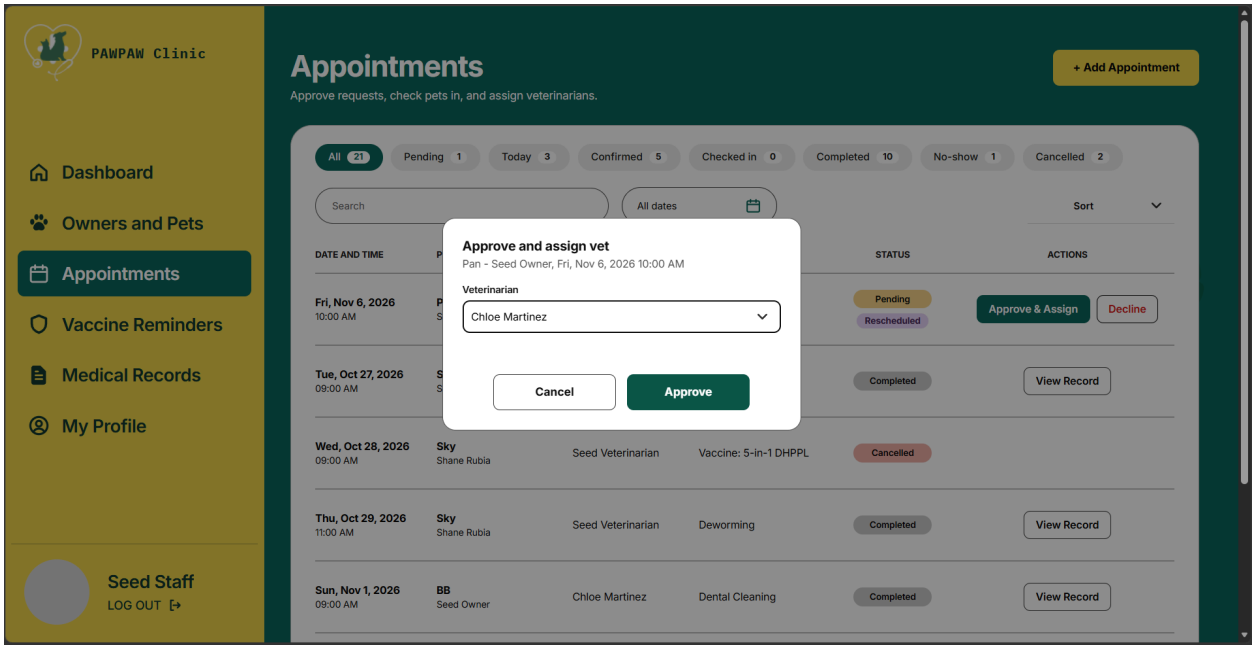
Create Register walk-ins and new pets or appointments.



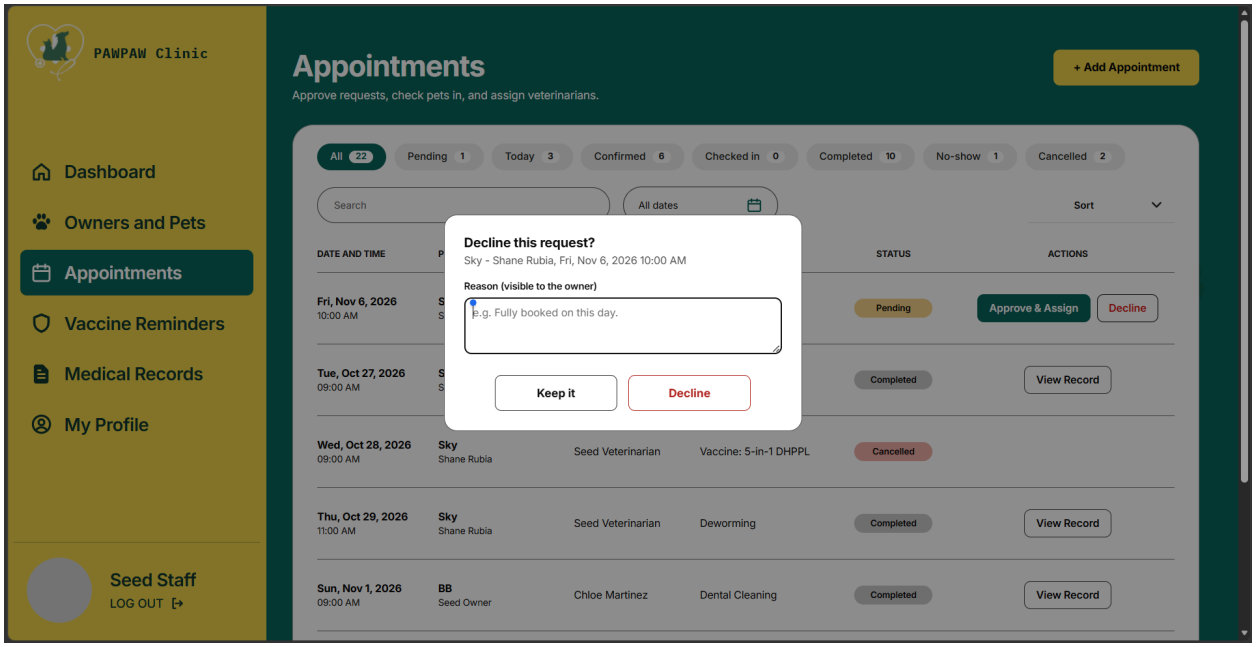
Read: View owner, pet, and appointment records.



Update : Update appointment details or status.

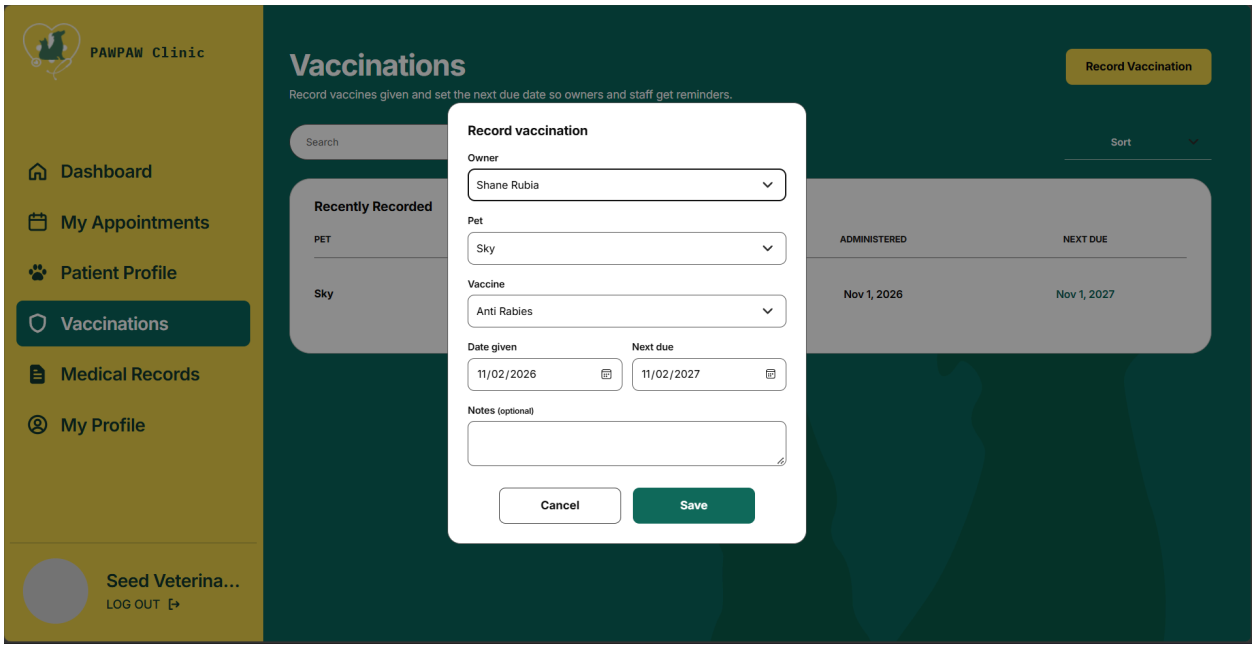


Delete: Show permitted cancellation.

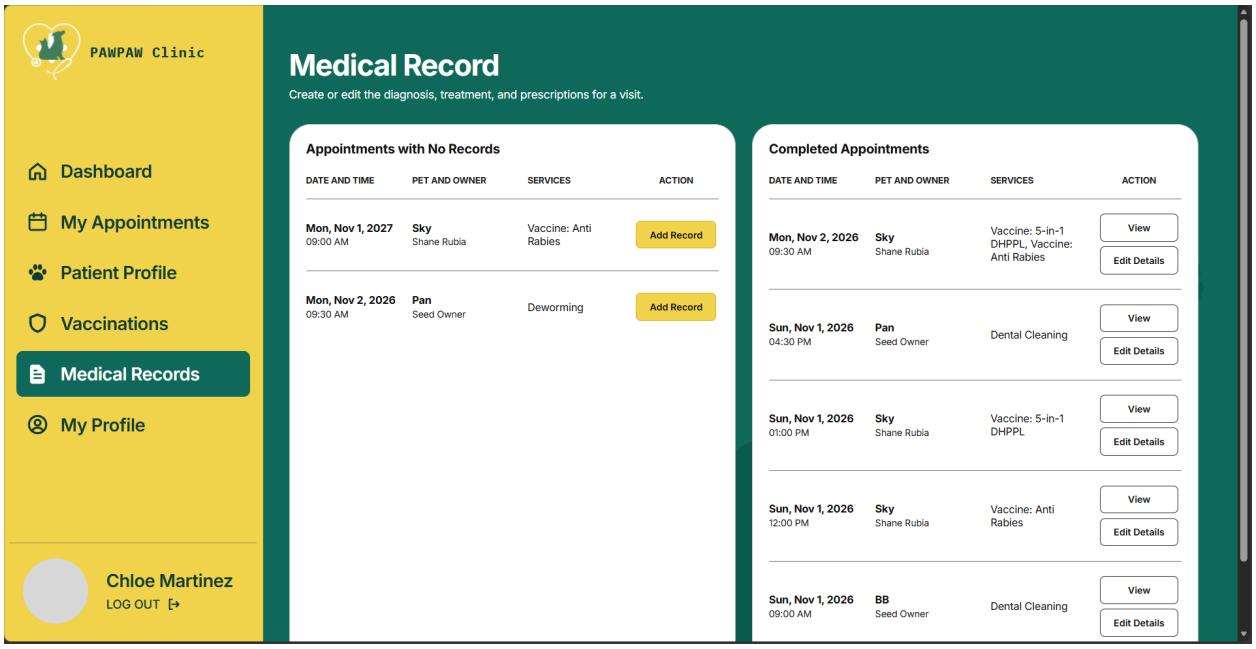


C. Veterinarian

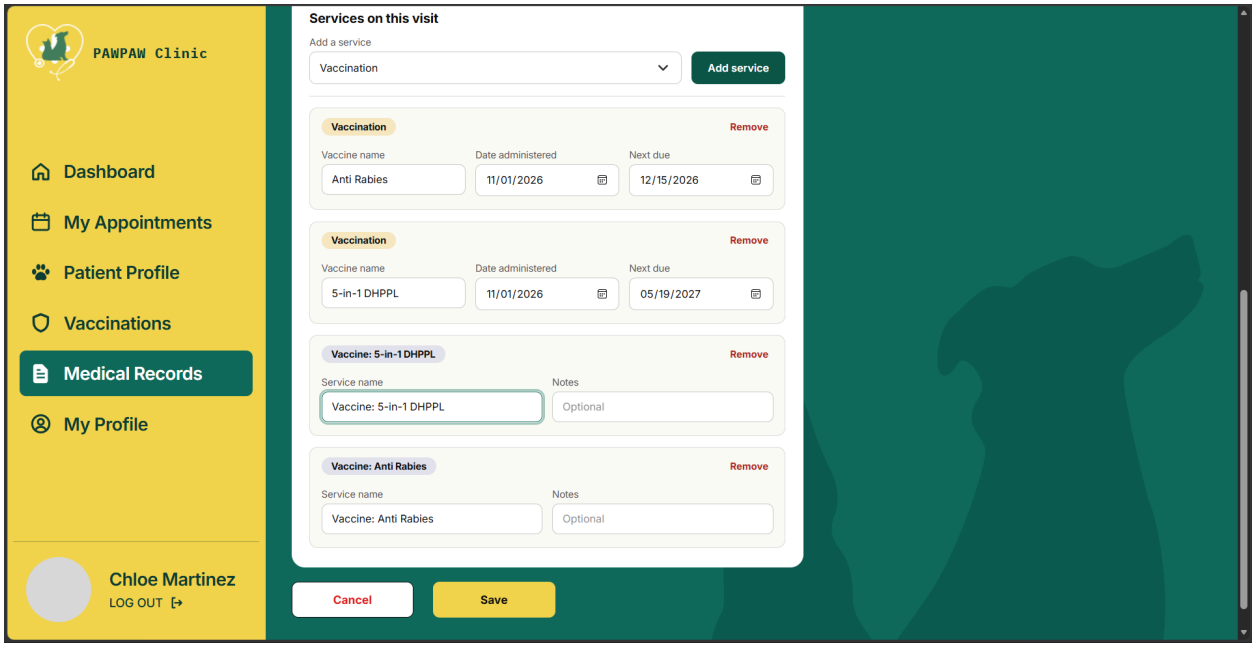
Create: Add a medical or vaccination record.



Read: View pet medical history.

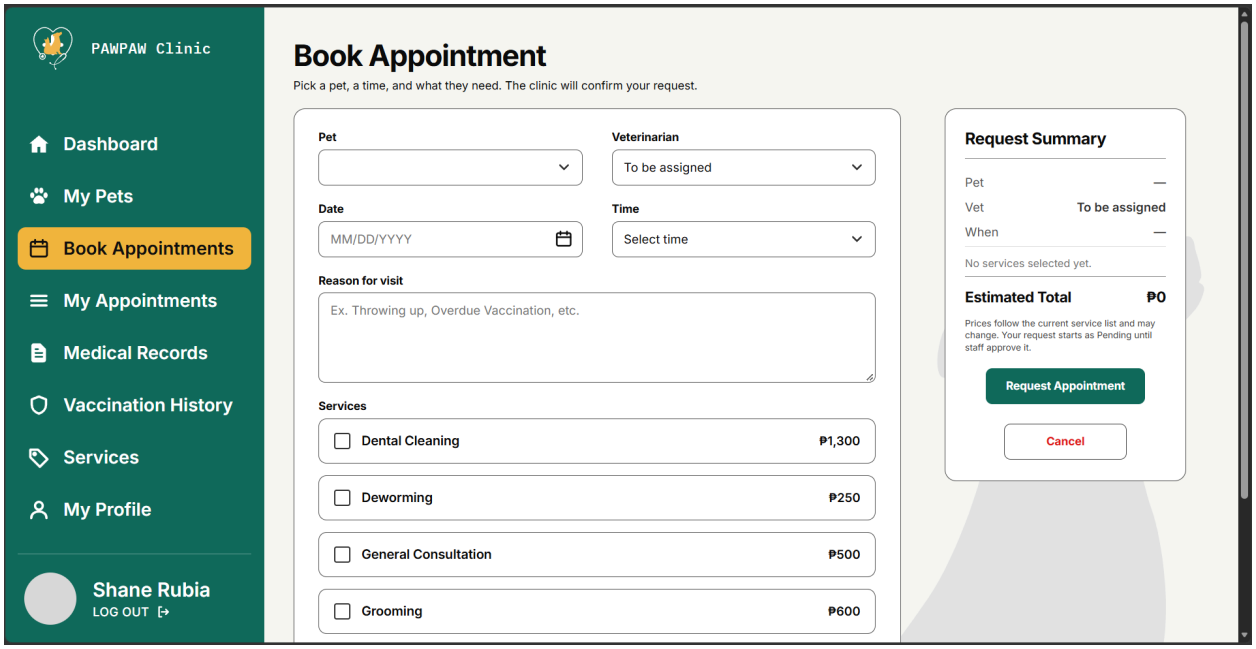


Update: Update medical or vaccination records.

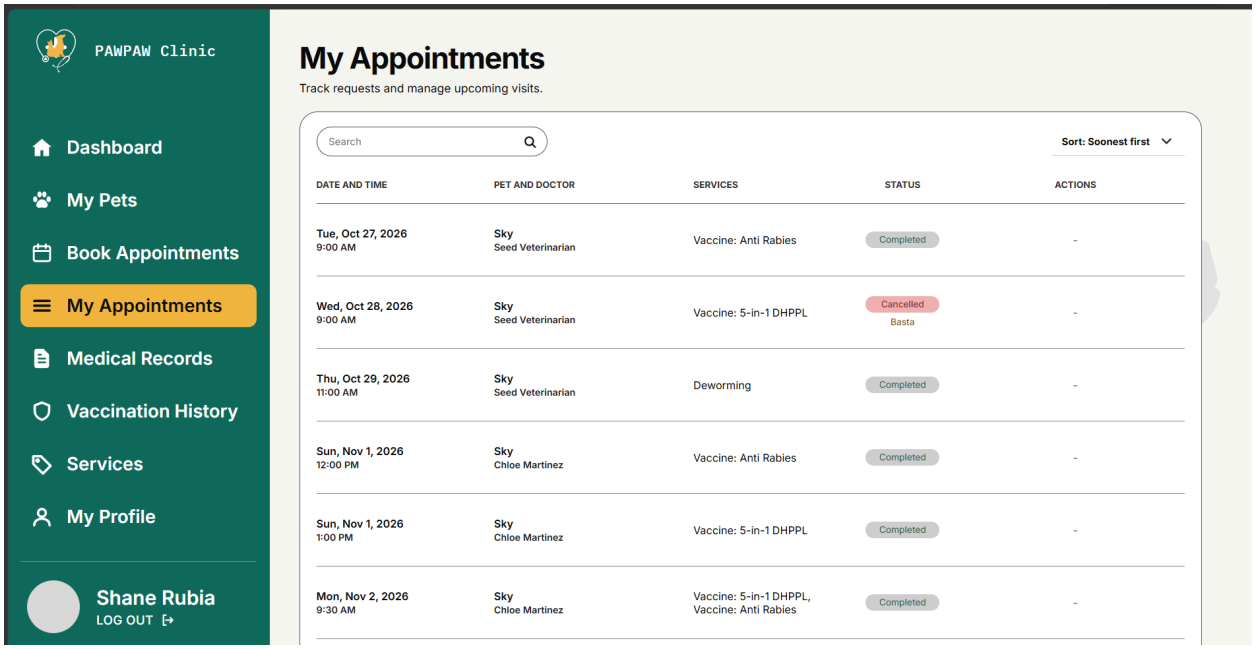


D. Owner

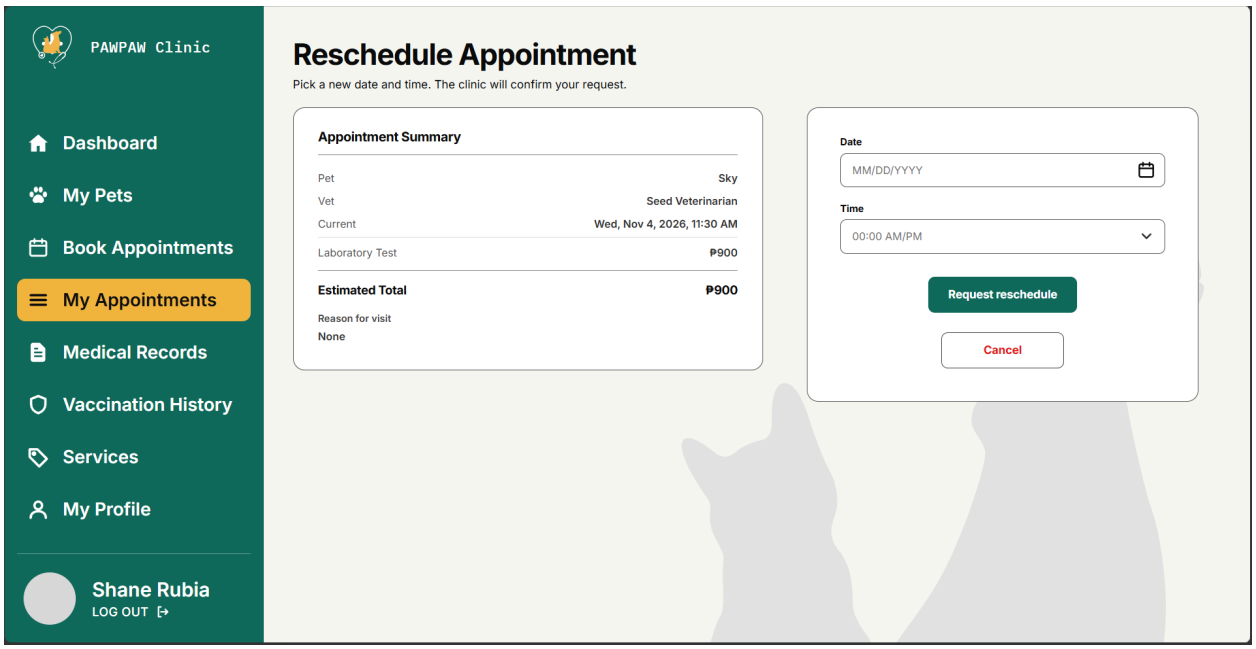
Create: Book a veterinary appointment.



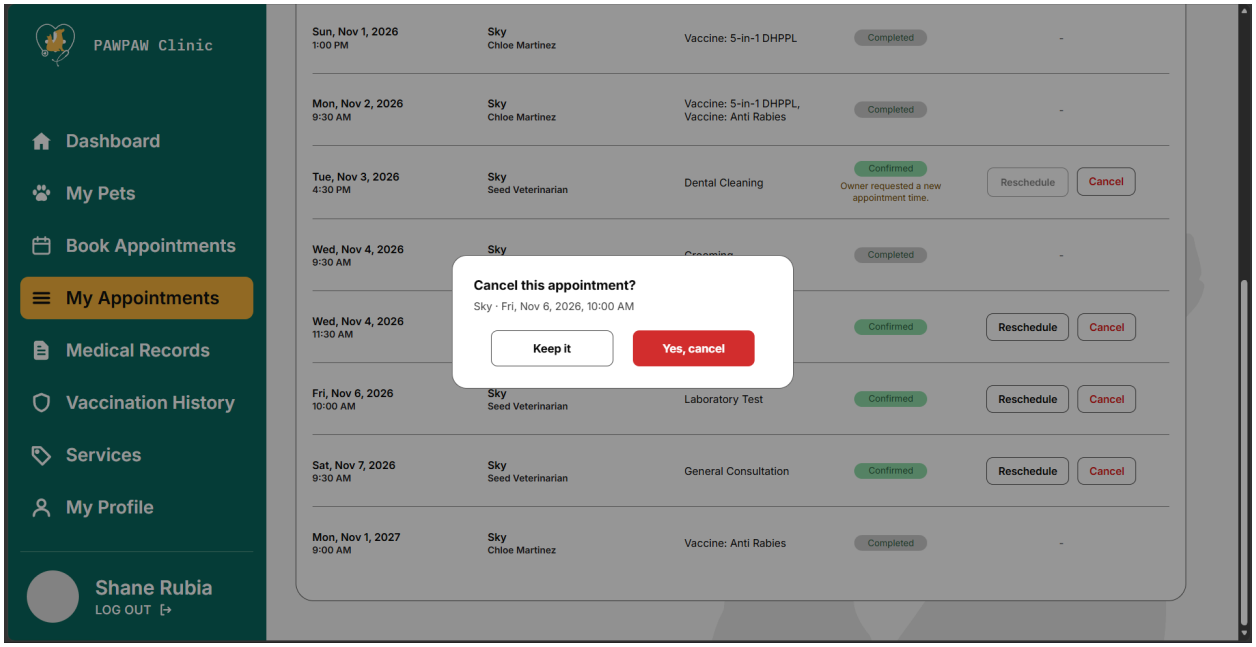
Read: View personal pets and appointments.



Update : Reschedule an eligible appointment.



Delete: Cancel an eligible appointment.



Setup and Configuration

This section outlines the installation procedures, system requirements, environment configuration, database initialization, and user roles of the Pawpaw Clinic Veterinary Clinic Management System.

1. **System Prerequisites**
   Before installing the application, the following software must be installed:

Software	Purpose
Node.js and npm	Backend runtime and package management
PostgreSQL	Relational database management system
Git	Repository cloning

2. **Installation and Setup Procedures**

Step	Procedure	Description
1	Clone the Repository	Clone the project repository from GitHub and navigate to the project directory.
2	Create the Database	Create a PostgreSQL database named group3_Pawpaw.
3	Configure Environment Variables	Navigate to the server directory, copy .env.example, rename the copy to .env, and configure the required application settings.
4	Install Dependencies	Install the required backend dependencies using npm inside the server directory.
5	Initialize the Database	Execute the database migration to create the required database objects using the schema file.
6	Create an Administrator Account	Create the initial administrator account using the credentials configured in the environment file.
7	Start the Application	Start the application server.
8	Access the Application	Open a browser and visit http://localhost:5000 to access the Pawpaw Clinic login page.

3. **Environment Configuration**
The application uses a .env file to manage its database connection and required application settings. The file is created by copying the .env.example template located in the server directory.

Environment Variable	Configuration / Placeholder
PORT	5000
DATABASE_URL	postgresql://USERNAME:PASSWORD@localhost:5432/pawpaw_clinic
AUTH_SECRET	your-random-secret-at-least-32-characters-long
ADMIN_USERNAME	admin
ADMIN_EMAIL	admin@example.com
ADMIN_NAME	System Administrator
ADMIN_PASSWORD	YourStrongPassword123!

Email and SMS credentials may also be configured if these integrations are intended to be used

4. **Database Configuration**
The system uses PostgreSQL as its relational database management system. The database schema is defined in the project's schema file and includes primary keys, foreign keys, and constraints to maintain data integrity.

Configuration	Details
Database Management System	PostgreSQL
Database Name	group3_Pawpaw
Database Schema File	server/schema.sql
Database Initialization File	server/migrate.js
Database Connectivity	node-postgres (pg)
Environment Configuration	server/.env

5. **Application User Roles**
The system supports four application user roles with specific responsibilities.

User Role	Responsibilities
Administrator	Manages users, staff, veterinarians, services, and clinic records.

Staff	Manages appointments, owner and pet information, walk-in registrations, and clinic records.
Veterinarian	Handles appointments, patients, medical records, and vaccinations.
Pet Owner	Manages pet profiles, appointments, medical records, and vaccination history.

6. **Setup and Configuration Troubleshooting**

Problem	Possible Solution
Database connection failed	Check PostgreSQL status and database credentials.
Port already in use	Change the PORT value in the environment file.
Missing environment variables	Check the environment configuration.
Email delivery error	Configure the required email credentials.
Dependencies missing	Install the dependencies inside the server directory.
Administrator creation failed	Verify the administrator environment variables and password requirements.

Issues and Resolutions

Stated here are problems encountered during development and setup, and how they were solved.

Date	Issue / Problem Encountered	Resolution / Action Taken	Status
October 18	Unclear business rules for registration, pet management, and appointment booking.	Reviewed the requirements and clarified the agreed workflows with the team before implementing the affected features.	Resolved
October 20	Database migration errors prevented the required table updates from being applied.	Checked dependencies and connection settings, verified the required tables and schema, and corrected the migration setup.	Resolved
October 22	Account role changes did not consistently align with related owner and veterinarian profiles.	Clarified the allowed role changes and updated account management logic to preserve related records and enforce profile requirements.	Resolved
October 24	Testing identified inconsistent access permissions for different user roles.	Corrected backend authorization checks and retested permitted and restricted actions for each role.	Resolved

October 26	Setup instructions did not reflect the latest database and backend changes.	Updated the README with the required environment settings, database preparation, migration, seeding, and startup steps.	Resolved

Cross-Group Verification

The group successfully cloned, configured, and tested the other groups' projects using separate local environments and PostgreSQL databases. Most applications were successfully installed and executed, although some required additional configuration due to complex setup instructions, unfamiliar technologies, and existing errors. These issues were documented in the Activity 3 Testing Log, and feedback was provided to the respective developers for improvement.

For our own project, Pawpaw, the feedback received was generally positive. The system was successfully reproduced, was easy to configure, and its essential functions worked as expected. Minor issues and recommendations were reviewed and addressed to improve the project's documentation, configuration, and overall reliability.

AI Usage Declaration

AI tools used, tasks performed with them, and how the group validated or corrected the output.

AI Tool	Purpose / Task	What the Group Changed / Verified	Member(s) Responsible
Gemini & Claude	Assisted in drafting and structuring the conceptual database model, entity relationships, and cardinalities for the ERD Assisted in developing the frontend, including page layouts, UI components, form handling, and connecting the interface to the backend API endpoints; also helped troubleshoot setup and runtime errors	The group reviewed, adjusted cardinality constraints, verified entity attributes against the business rules, and integrated the final ERD design. The group reviewed and edited the generated code to match the approved UI/UX design, tested each page and form against the system's user roles (Pet Owner, Staff, Veterinarian, Admin), verified that data displayed and saved correctly through the API, and fixed issues found during testing.	Anajoy

ChatGPT	Assisted in reviewing and validating the system scope, functional requirements, and alignment of implemented features with the project's objectives and business rules. AI was also used to review documentation for accuracy, consistency, completeness, and clarity, as well as to identify possible gaps, inconsistencies, and areas for improvement.	The group independently reviewed and verified the AI-assisted suggestions against the approved project scope, system requirements, actual functionalities, and database design. Necessary revisions were made to ensure that the implemented features followed the established requirements and that the documentation accurately reflected the completed system. All final decisions, modifications, and validations were performed by the group.	Roxanne
ChatGPT & Claude	ChatGPT assisted in finalizing and refining the database schema, reviewing the ERD, and verifying entity attributes, relationships, primary keys, foreign keys, and constraints. Claude assisted in troubleshooting backend issues, particularly database connectivity, API-to-database integration, and server-side errors.	We reviewed and refined the database schema to ensure consistency with the ERD and system requirements. AI-assisted backend implementations were integrated, tested, and adjusted to support key system functionalities, including appointment booking and rescheduling, vaccination record management, and user account management. We also verified database connectivity, API communication, business rules, and proper data handling to ensure these features worked as intended.	Shane
Claude	Assisted in generating a starting point for some UI/UX elements and the baseline page layout, including the basic structure and arrangement of components, which was	the AI-generated layout was used only as a starting point. It was reviewed, modified, and customized to match the approved UI/UX design and the system’s requirements. The elements were tested to confirm they displayed	Sean

	then used as a foundation for the interface	correctly and worked as intended, and any inconsistencies in spacing, styling, or functionality were fixed manually.	